

Práctica 2. Estudio y propuesta de paralelización de una aplicación

1. Objetivos

En esta práctica seleccionareis una aplicación secuencial cuyo coste computacional justifique estudiar su paralelización. Analizareis su estructura, medireis su rendimiento y propondreis cómo podría ejecutarse en una arquitectura paralela. La implementación paralela se abordará en prácticas posteriores.

Al finalizar, deberíais ser capaces de:

- Identificar aplicaciones con una carga computacional adecuada para su paralelización.
- Reconocer dependencias entre tareas y oportunidades de paralelismo de datos y funcional.
- Estudiar cómo varía el rendimiento al cambiar el tamaño o la complejidad del problema.
- Medir el efecto de distintas opciones de compilación e identificar posibles casos de autovectorización.
- Proponer una arquitectura paralela apropiada y razonar qué ganancia y eficiencia cabría esperar.

2. Desarrollo

Tarea 1. Selección de la aplicación

Buscad o diseñad un problema cuya solución requiera una aplicación de coste computacional apreciable. Puede proceder de cualquier ámbito: ingeniería, matemáticas, física, química, biología, ciencias sociales o cualquier otra disciplina.

En esta segunda práctica trabajaréis con una implementación secuencial del problema, escrita en C o C++. Podéis desarrollar el código vosotros mismos o partir de un proyecto existente, siempre que identifiquéis y citéis su procedencia y cualquier modificación que realicéis.

No necesitáis dominar todos los fundamentos de la disciplina a la que pertenece el problema, pero sí comprender qué hace el programa, cómo está organizado y de qué depende su tiempo de ejecución.

La aplicación debe poder ejecutarse **sin intervención durante la medición**: recibirá los parámetros al inicio, mediante argumentos o un archivo de configuración, y producirá los resultados al terminar. Si elegís una aplicación originalmente interactiva, tendréis que definir una forma reproducible de ejecutarla sin intervención; por ejemplo, enfrentando dos instancias de un programa de ajedrez.

Elegid un caso de prueba de referencia que tarde, como mínimo, del orden de segundos en una ejecución secuencial, preferiblemente entre varios segundos y unas decenas de segundos. Podéis trabajar con casos mayores si resultan manejables. Evitad configuraciones que terminen en microsegundos o nanosegundos: en ellas será difícil medir con claridad las mejoras y valorar el coste adicional de crear, comunicar y sincronizar hebras o procesos. Indicad en la memoria los parámetros del caso de prueba y su tiempo de ejecución.

Buscad una aplicación en la que podáis identificar **paralelismo de datos y paralelismo funcional**:

- Hay *paralelismo de datos* cuando una misma operación puede aplicarse simultáneamente a partes distintas de los datos.
- Hay *paralelismo funcional* cuando distintas operaciones o tareas pueden ejecutarse simultáneamente, siempre que sus dependencias lo permitan.

Una multiplicación de matrices simple es un buen ejemplo de paralelismo de datos, pero para esta práctica buscad un problema cuya estructura también permita estudiar tareas diferentes que puedan solaparse.

Organización de la tarea. Dedicad las primeras 2–3 horas de prácticas a buscar y valorar candidatos. Comentad con vuestro profesor o profesora las opciones que estéis considerando antes de continuar con el análisis.

Al finalizar la segunda sesión de prácticas, cada equipo comunicará el título y una breve descripción del problema elegido, junto con la lista definitiva de sus integrantes. Esta información se publicará en **Moodle/UACloud** (según la preferencia del profesorado).

Podéis consultar artículos, libros, repositorios de código y publicaciones de las Jornadas SARTECO, entre otras fuentes. Las contribuciones a las Jornadas de Paralelismo o a las Jornadas de Computación Empotrada y Reconfigurable pueden daros ideas que adaptar a esta práctica: www.jornadassarteco.org.

Tarea 2. Estudio de la aplicación secuencial

Analizad el código que habéis elegido con suficiente detalle para explicar **por qué es un buen candidato para la paralelización y qué dificultades plantea**. Apoyad vuestras conclusiones en mediciones, gráficas y fragmentos concretos del programa.

2.1. Tareas y dependencias

Dividid la aplicación en sus tareas principales y representadlas mediante un **grafo de dependencias**. Indicad qué tarea debe terminar antes de que comience otra y cuáles podrían ejecutarse a la vez sin cambiar el resultado.

Por ejemplo, un programa puede tener una fase de inicialización seguida de dos cálculos independientes cuyos resultados se combinan al final. Mostrad esa estructura en el grafo y relacionad cada tarea con la parte del código que la implementa.

2.2. Datos y accesos a memoria

Identificad las estructuras de datos y variables más relevantes. Explicad cuáles se consultan o modifican con frecuencia, cuáles se utilizan solo en fases concretas y cómo accede a ellas el programa.

Estudiad si una organización diferente de los datos o del flujo de ejecución podría favorecer una futura versión paralela. Comentad también los posibles problemas relacionados con la jerarquía de memoria, especialmente el uso de la caché y los accesos compartidos entre tareas.

2.3. Tamaño del problema y carga de trabajo

Identificad uno o varios parámetros que permitan variar la carga de trabajo: por ejemplo, las dimensiones de una matriz, el número de elementos procesados o la precisión requerida.

Ejecutad la **versión secuencial** con distintos valores y representad gráficamente cómo cambia su tiempo de ejecución. Indicad las condiciones en las que habéis realizado las mediciones y explicad las tendencias observadas.

En este apartado estudiáis cómo escala el **tiempo de la aplicación secuencial**. La ganancia en velocidad y la eficiencia de una versión paralela se podrán medir cuando exista dicha versión.

2.4. Opciones de compilación y autovectorización

Compilad y ejecutad la aplicación en el entorno Linux indicado para la asignatura, utilizando GCC (gcc o g++).

Seleccionad varias opciones de compilación que puedan afectar al rendimiento, explicad brevemente qué hace cada una y recoged en una tabla los tiempos obtenidos. Realizad las comparaciones con el mismo caso de prueba y en condiciones lo más parecidas posible. Indicad siempre los comandos de compilación utilizados.

Buscad también evidencias de **autovectorización** en alguna parte del programa: comprobad si el compilador genera instrucciones SIMD y señalad a qué parte del código corresponden. Podéis consultar los informes de optimización del compilador y, cuando sea útil, inspeccionar el código ensamblador generado.

Como punto de partida, consultad la [documentación oficial de GCC](#) y la ayuda de gcc --help=optimizers y gcc --help=target —o sus equivalentes para g++—.

2.5. Comparación de configuraciones de compilación

Comparad las combinaciones de opciones que hayáis probado. Representad gráficamente sus tiempos de ejecución y, si resulta útil, calculad la ganancia respecto a una **compilación de referencia** que identifiquéis claramente.

Explicad por qué creéis que unas configuraciones funcionan mejor que otras. Relacionad vuestra interpretación, cuando corresponda, con el uso de la memoria y la caché, la vectorización o la generación de instrucciones específicas para el procesador.

2.6. Interpretación del rendimiento

Reunid los resultados de los apartados anteriores y explicad cómo afectan al rendimiento tanto el tamaño del problema como la configuración de compilación.

Identificad qué partes de la aplicación parecen limitar más su tiempo de ejecución y cuáles ofrecen mejores oportunidades de mejora. Distinguid entre **mejoras medidas en la aplicación secuencial** y **ganancias paralelas estimadas**, que todavía no habéis medido.

Tarea 3. Preparación para las siguientes prácticas

Resolved los siguientes problemas. En cada caso, calculad el tiempo necesario para llenar el depósito, la ganancia en velocidad respecto al uso del grifo más rápido de los disponibles en ese caso y la eficiencia respecto al número de grifos utilizados. Indicad las expresiones empleadas y explicad brevemente los resultados.

1. Un grifo tarda 8 horas en llenar un depósito y otro tarda 40 horas. ¿Cuánto tardarán si abrimos los dos a la vez para llenar el depósito, inicialmente vacío?
2. Disponéis de dos grifos, cada uno de los cuales tarda 8 horas en llenar el depósito. Repetid los cálculos.
3. Disponéis de dos grifos, cada uno de los cuales tarda 40 horas en llenar el depósito. Repetid los cálculos.
4. Disponéis de tres grifos: uno que tarda 8 horas y dos que tardan 40 horas. Repetid los cálculos.



Suponed que todos mantienen un caudal constante y que sus caudales se suman cuando se utilizan simultáneamente.

Tarea 4. Memoria y archivos que debéis entregar

Entregad una **memoria estructurada** que permita seguir vuestro trabajo y reproducir las mediciones. incluid, como mínimo:

1. Objetivos y descripción del problema elegido.
2. Procedencia del código y modificaciones realizadas, si las hay.
3. Descripción de la aplicación y del caso de prueba de referencia.
4. Grafo de dependencias y análisis de las oportunidades de paralelismo de datos y funcional.
5. Estudio de los datos y de los accesos a memoria.
6. Metodología de medición, tablas, gráficas y análisis del rendimiento secuencial.
7. Opciones de compilación probadas y evidencias de autovectorización.
8. Justificación de por qué la aplicación es una buena candidata para su paralelización.
9. Propuesta razonada de una o varias arquitecturas paralelas adecuadas —por ejemplo, un procesador multinúcleo, una GPU, un clúster o un sistema heterogéneo—, junto con sus posibles ventajas y dificultades.
10. Resolución de los problemas de la tarea 3.
11. Conclusiones y bibliografía.

Entregad también el código fuente y un archivo Makefile que permita ejecutar, al menos, los siguientes comandos:

- make: compilar el proyecto.
- make run: ejecutar la aplicación con los parámetros del caso de prueba de referencia.
- make clean: eliminar los archivos generados durante la compilación.

La entrega se realizará mediante **UACloud**, **antes de la sesión de prácticas de la semana del 12 de octubre**. Dispondréis de **TRES sesiones** para trabajar en esta práctica. La actividad de entrega indicará la fecha y hora exactas y los formatos admitidos.

NOTA: Debido al festivo del 9 d'octubre, la entrega se retrasará en el grupo de prácticas de los viernes.

Originalidad. El trabajo que presentéis debe ser vuestro. Podéis utilizar código y otras fuentes externas si su uso está permitido, los citáis y explicáis qué habéis aportado o modificado. Las entregas se evaluarán conforme a las normas de integridad académica aplicables en la Universidad de Alicante.